

US-MADE AGENT MEMORY

STRATUS

Four-Tier Memory

What it is · How it works · Scaling to hundreds of agents

Apache-2.0 · github.com/rick-stevens-ai/stratus · Kukla · June 2026

OVERVIEW

What STRATUS is

A clean-room, Apache-2.0 memory engine: it turns raw agent conversation into durable, searchable knowledge — automatically.

Raw conversation



Distilled memory



Recalled in context

Two deps only

better-sqlite3 + sqlite-vec. No cloud.

Self-contained

One SQLite file + markdown blobs.

Runs offline

Deterministic local embedder built in.

Why agents need this

The problem

Stateless agents wake up blank. Context is finite.
Monday's knowledge is gone by Tuesday.

The naive fix

Dump it all in the prompt. Blows the budget, drowns the signal.

The STRATUS answer

A tiered hierarchy — like cache → RAM → disk.

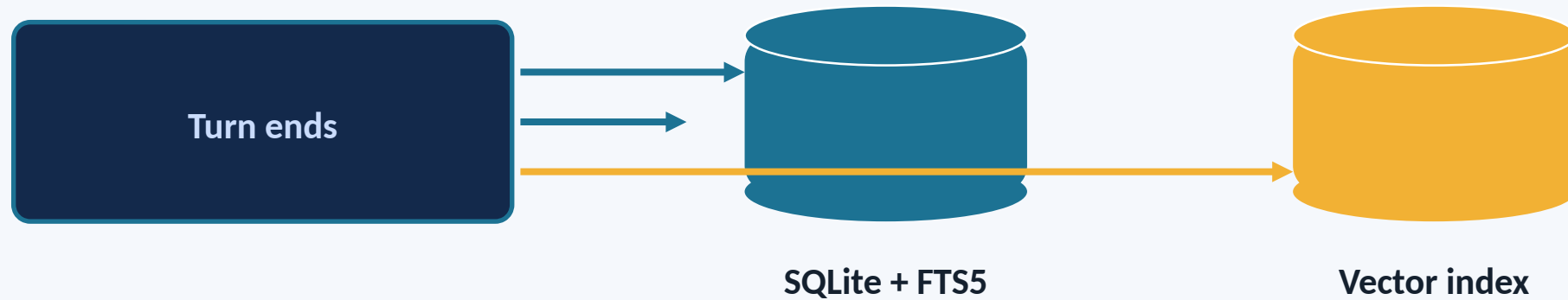
Hot memory is injected. The long tail stays searchable on demand.

The agent carries a compact working set, not its whole past — and the bits stay on infrastructure we control.

The four tiers



To — the conversation stream



Lexical + semantic in one store

Full-text (FTS5) and vectors (sqlite-vec) live side by side
— no separate services.

Live on this machine

579 turns captured and full-text indexed, mirrored from
the running agent in shadow mode.

T1 — atoms (extracted facts)

On a schedule, an LLM distills the stream into durable, typed atoms:

persona

Stable traits & preferences.

“Wants LLM judgment, not regex.”

episodic

Events, with dates.

“Fixed the FTS5 500 bug on 06-23.”

instruction

Standing rules.

“Never echo passwords.”

Each atom carries a priority and is upserted — duplicates merge instead of piling up.

T2 scenes → T3 core

T2 — Scenes

Related atoms grouped into a readable narrative chunk, written as a dated markdown blob.

“Converging STRATUS with TDAI for shadow dual-run.”

A middle unit between one fact and the whole history.

T3 — Core

From all scenes, an LLM synthesizes one profile (core.md), updated continuously.

The most-compressed tier — injected first on every recall.

The agent's standing model of its human.

MECHANICS

The promotion pipeline

Tiers fire in sequence, in a background sidecar — the conversation never waits.



Capture is instant; distillation is a separate process. A STRATUS bug can never slow or break the agent.

INTERFACE

One gateway, four tiers

Everything is reachable over a small HTTP API on :8077 — one route group per tier.

/stream/*	add · search · query
/atoms/*	upsert · search · query
/scenes/*	write · read · ls
/core/*	write · read

Also shipped: a CLI (bin/stratus) and a 13-test smoke suite — “ALL TIERS GREEN.”

How memory is searched

Lexical

FTS5 exact-term match. Fast, transparent.

Semantic

sqlite-vec meaning match.
Related ideas, different words.

Hybrid

Blends both. The default.

Two embedder modes — same code

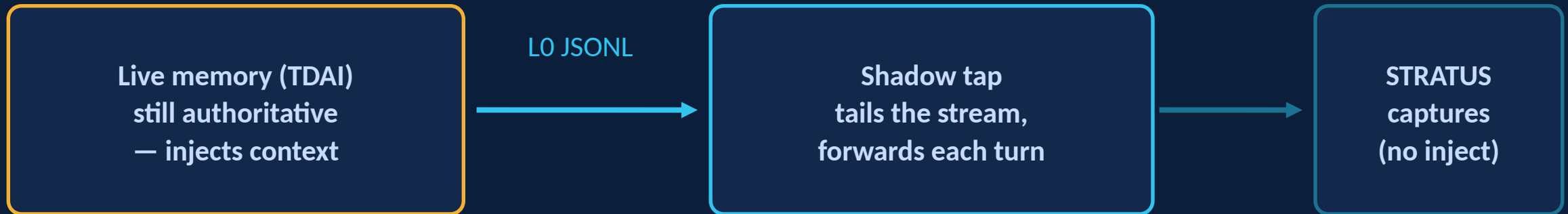
STRATUS_EMBED=local — deterministic, offline, zero network.

STRATUS_EMBED=remote — any OpenAI-compatible endpoint (we use Argo / ALCF).

VALIDATION

Proving it safely: shadow mode

STRATUS runs alongside the live system for a week — capturing, never injecting.



Why it's safe

STRATUS never feeds the agent loop during the trial. A bug degrades nothing. A weekly compare job grades capture parity, tier counts, and recall A/B before any cutover.

ON THIS MACHINE

STRATUS, live in shadow

579

T0 turns

210

T1 atoms

13/13

smoke green

2

runtime deps

What this means

From 579 mirrored turns the sidecar distilled 210 atoms, a scene blob, and a synthesized core profile — on two npm dependencies, one SQLite file, and no cloud.

PART TWO

Scaling to hundreds of agents

Three tiers that scale independently

From one store to a fleet



Agents never block on each other; the queue absorbs load and preserves order.

The gap, and the plan

Today: one store per process

STRATUS ships single-store. Per-agent isolation — a tenant key routing each request to its own DB + blob path — is the one piece not yet built. Everything else already exists.

- 1 Tenant routing** Key each request to its own store — the missing piece.
- 2 Externalize storage** Single-file → pooled / clustered behind one adapter.
- 3 Distribute the distiller** Sidecar → autoscaling worker pool.
- 4 Load-test to hundreds** Prove latency + parity under full fleet load.

SUMMARY

Bottom line

What it is

A US-made, Apache-2.0, four-tier memory engine.

How it works

T0→T3 distillation; one HTTP gateway; lexical+semantic search.

Why trust it

2 deps, runs offline, validated in shadow against the live system.

Scaling it

Three independent tiers; only tenant-routing remains to build.